

Intro to Local LLMs



Kei Tang

2026-09-25

Recurse Center



1. Introduction	2
2. Model sizes and memory: What model can I run?	6
3. GPUs and optimizations: How fast will the model run?	12
4. Model repositories and Inference frameworks	16
5. Demo	19

1. Introduction





- **Privacy:** Cloud providers may retain your data or use it for training
- **Cost:** Powerful cloud models are expensive
- **Research and learning:** Know more about how LLMs work
- **Ethics:** Avoid the big model providers controlling the technology
- and more?

1.2 Local LLMs are quite powerful now

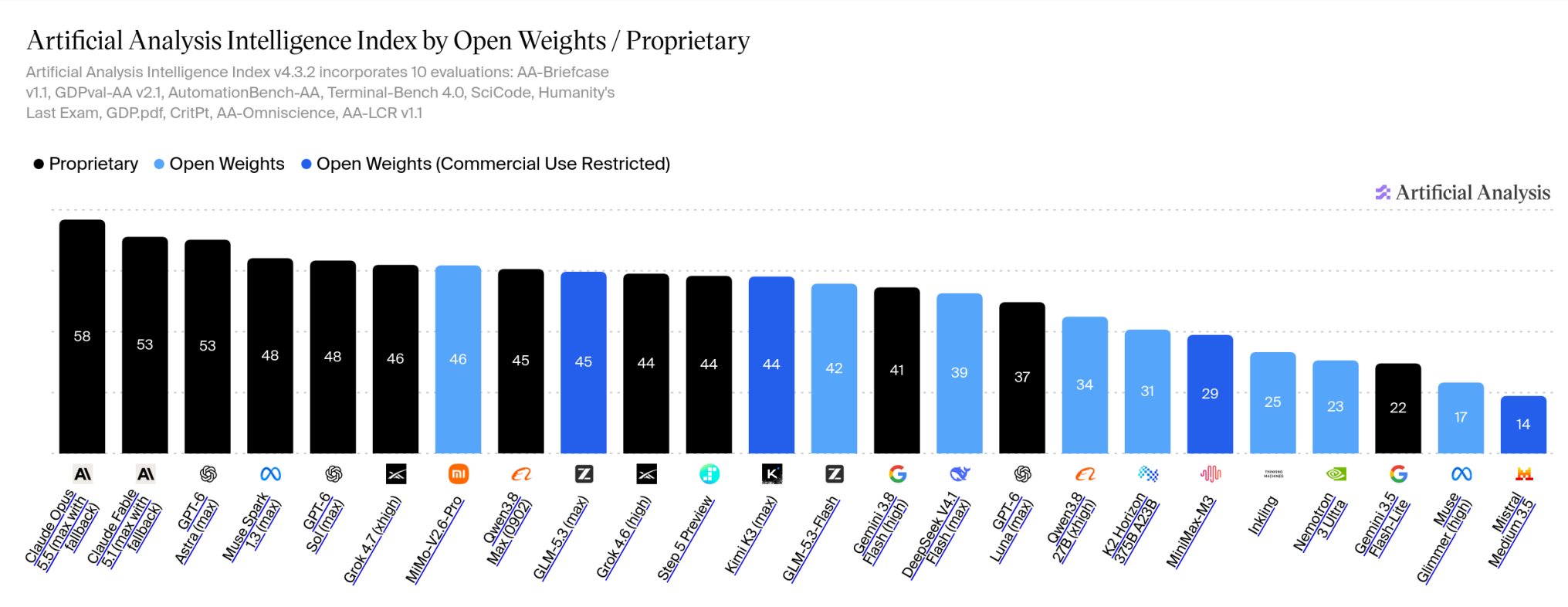


Figure 1: The AA Intelligence Index. (Take all benchmark rankings with a grain of salt!)



- **Harness**: Models are generally decoupled from harnesses
- **Training**: Maybe another workshop in the future?
- **Platform specific settings (e.g. Metal for Macs, Nvidia stuff)**: I don't own those hardware, but we can look into it together!

2. Model sizes and memory: What model can I run?



Model size for language models are usually characterized by **number of parameters** (in billions):

- The scaling era: GPT-2: 1.5B, GPT-3: 175B, GPT-4: 1.8T (unconfirmed)
- Best open models: ~1T
- With consumer hardware, 100B is usually the limit (...Why?)



2.2.1 Which “memory”?

(Usually) LLMs are deployed on GPUs, so the model needs to fit on the VRAM.

- **Exceptions:** Newer Macs, some AMD chipsets, (and game consoles) use *unified memory*, which means that system RAM can work as VRAM.



2.2.1 Which “memory”?

(Usually) LLMs are deployed on GPUs, so the model needs to fit on the VRAM.

- **Exceptions:** Newer Macs, some AMD chipsets, (and game consoles) use *unified memory*, which means that system RAM can work as VRAM.

2.2.2 Deriving memory requirement from model size

- **Model parameters:** $\text{memory} = \text{number of parameters} * \text{bits per parameter}$
 - By default, LLMs use FP16 (16bit). A 100B model will consume $100\text{B} * 16\text{bit} = 200\text{GB}$.
- **Dynamic memory usage:** Additional memory is required to store intermediate computation values (mostly KV cache).
 - This part *grows with context length*. When a large context length is required, this extra memory can be as large as $0.5 * \text{model size}$.

2.3 Quantization



Quantization takes an existing model, reduces memory usage by changing the numeric data structure for parameters, usually with little drop in accuracy.

- Classic quantization methods use INT8 or INT4.
- Recent methods dynamically select some parameters to quantize in order to minimize accuracy loss. *Quants* range from 1 to 8 bits.
- eg. $100\text{B} * 4\text{bit} = 50\text{GB}$

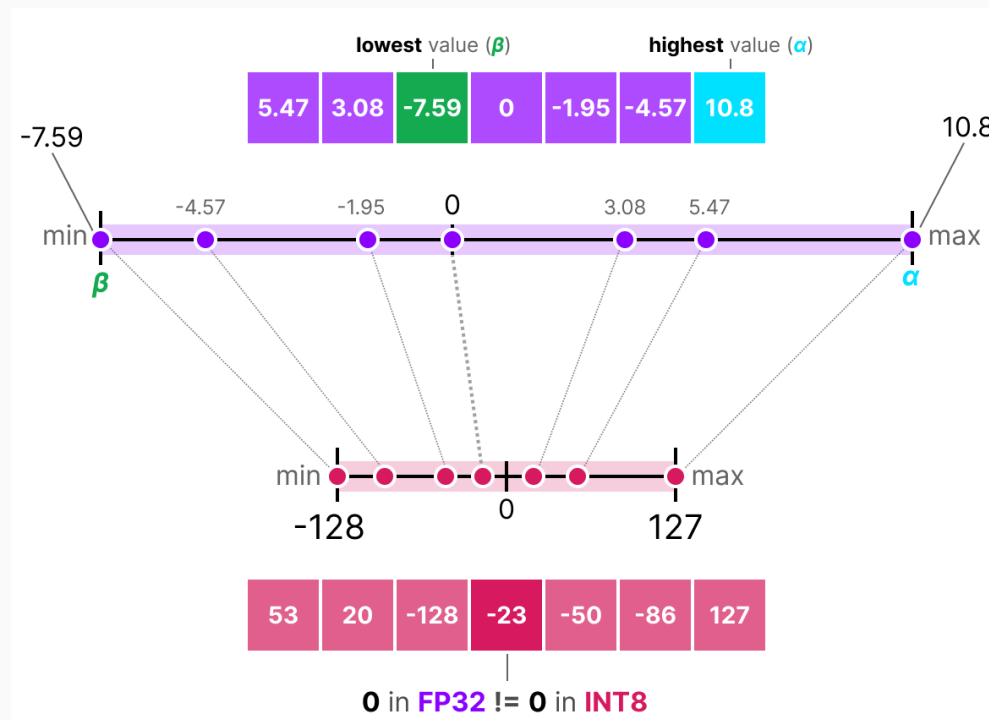


Figure 2: From FP32 to INT8¹

¹Image credit: <https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-quantization>

2.3 Quantization

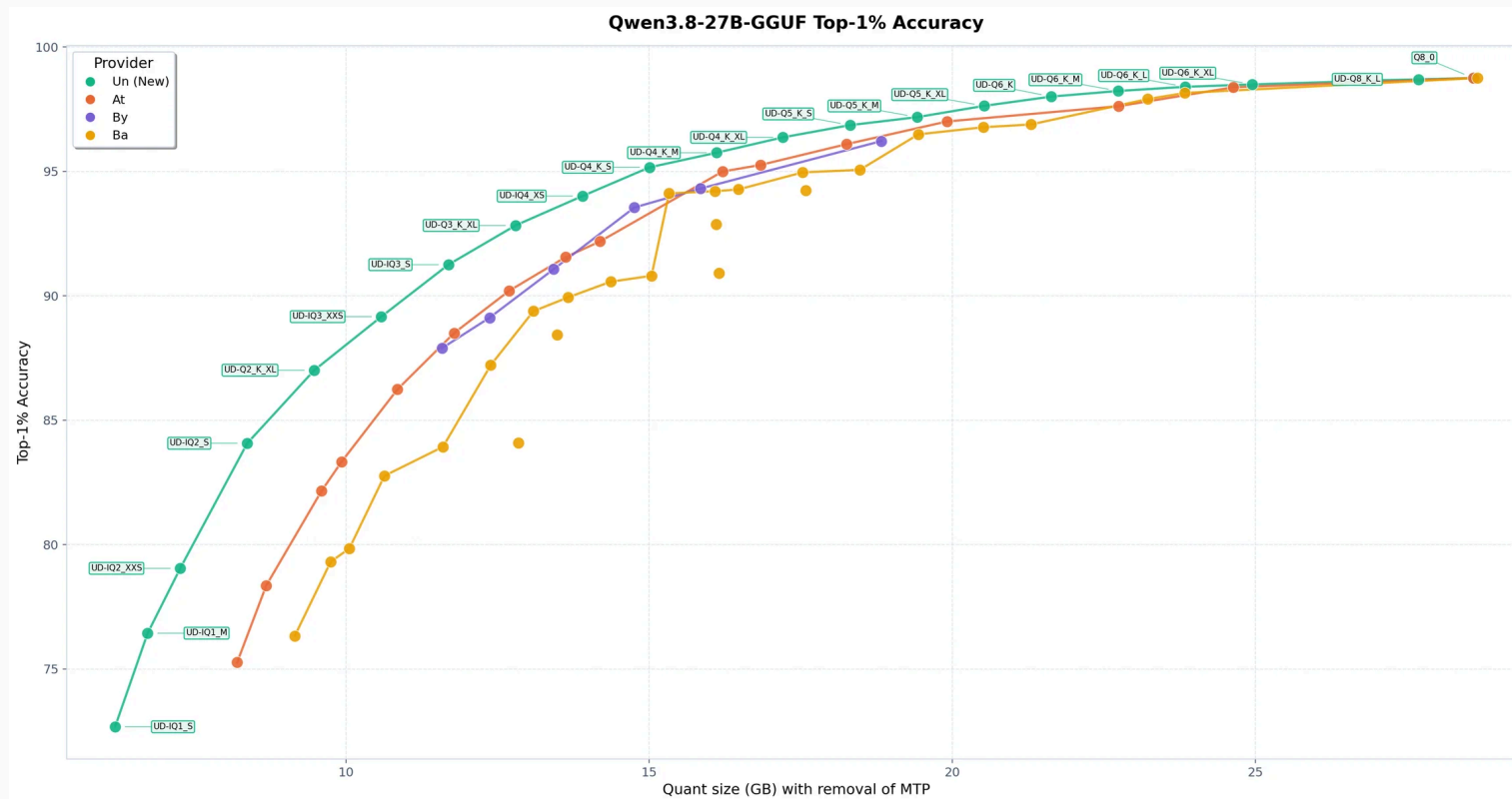


Figure 3: The different Qwen3.8-27B quants (X: model size, Y: Top-1 accuracy)¹

¹<https://unsloth.ai/docs/basics/dynamic-3.0-ggufs>



As a general rule of thumb:

- Memory (in GB) $\sim$ the size (in B) of model you can run comfortably when quantized

3. GPUs and optimizations: How fast will the model run?

3.1 How “speed” is measured



3.2 Factors affecting “speed”





4. Model repositories and Inference frameworks

4.1 Where to find & download models?



4.2 All the different frameworks!



5. Demo





1. Choosing a (quantized) model
2. Running it in llama.cpp
3. Running it in Ramalama